

ATOSFleetManagement: TruckObjectControl Overview

File: atos/modules/TruckObjectControl/src/truckobjectcontrol.cpp
Generated: 2026-04-27

1) Runtime Flow

1. Node startup reads parameters, loads trajectory GeoJSON, starts TCP/TLS listener, and starts a 200 ms timer.
2. CoT arrives from either TCP/TLS clients or ROS topic truck_objects/cot.
3. Parser extracts uid, lat/lon, speed, course, optional path_name/path_index.
4. Position is projected to distance along trajectory; state is updated in trucks_[uid].
5. GUI state is published on truck_objects/state, including latest CoT (Rx), latest TCP (Tx), warnings.
6. Every 200 ms, controller groups trucks per path and computes next-ahead gaps.
7. Per-truck command is decided: SLOWDOWN / STOP / RESUME.
8. Command is sent back on same TCP/TLS connection and fleet summary is published to truck_objects/speed_command.

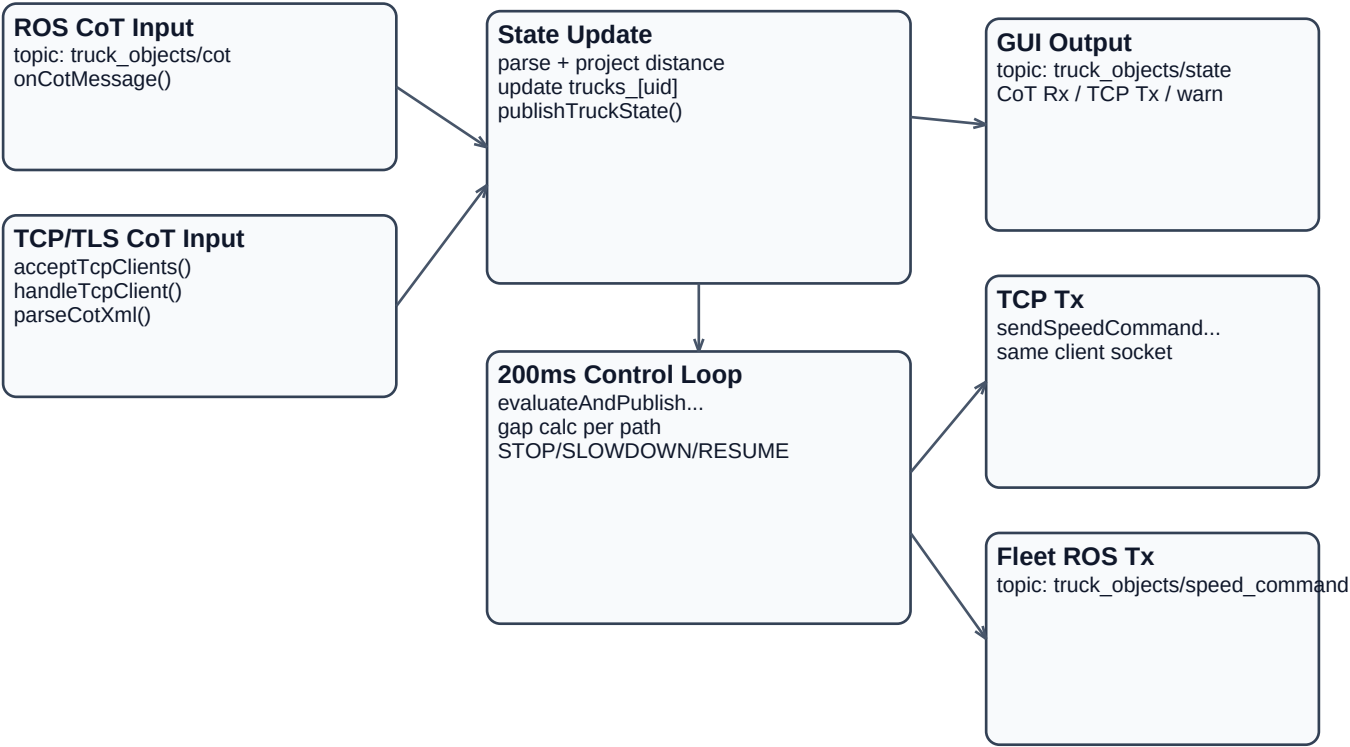
2) Gap-based Decision Logic

Condition	Action
500m >= X > 200m	SLOWDOWN to warning_speed_kmh/3.6 m/s
X <= 200m	STOP to stop_speed_kmh/3.6 m/s
X > 300m and previous was STOP	SLOWDOWN
X > 700m and previous was SLOWDOWN	RESUME

3) TCP Command Format (per truck)

```
command=<STOP|SLOWDOWN|RESUME>;
target_speed_mps=<value>;distance_to_truck_ahead_m=<value>;
truck_ahead_path_index=<idx>;truck_ahead_uid=<uid>;
reason=<text>;min_gap_m=<value>;connected_count=<N>;
path_name=<name>;cmd_seq=<seq>
```

TruckObjectControl Architecture Diagram

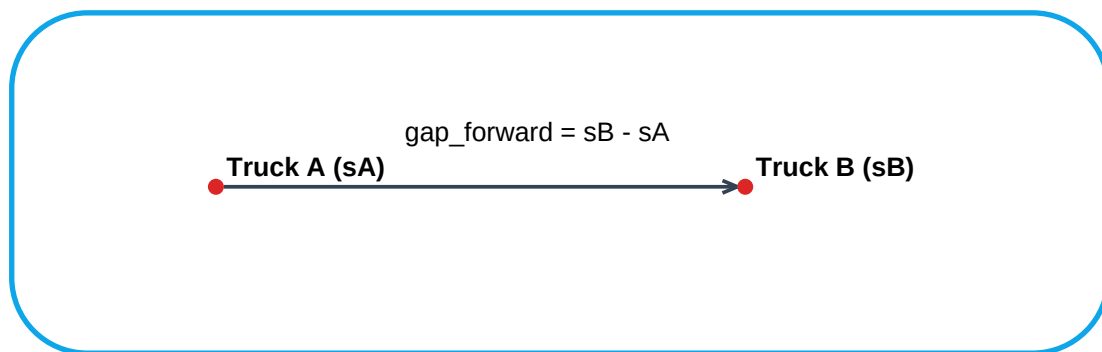


Note: TCP_NODELAY is enabled on accepted sockets for immediate command transmission.

Distance Gap Calculation (Vincenty-based)

Description

ATOS stores path points $P_0..P_n$ with cumulative station $s(i)$ in meters. Segment lengths are geodesic distances between consecutive lat/lon points. For each truck, current lat/lon is projected to trajectory and converted to station s_{truck} . For truck A and next truck ahead B on same path: $gap = s(B) - s(A)$. If $gap \leq 0$, wrap with $path_length$. This gives circular forward distance used by STOP/SLOWDOWN/RESUME logic.



Wrap-around case:

```
if (sB - sA) <= 0:    gap = (sB - sA) + path_length
else:                 gap = (sB - sA)
```

Vincenty Algorithm Details in This Implementation

What Vincenty computes

Vincenty inverse computes geodesic distance on the WGS84 ellipsoid between two points (lat1, lon1) and (lat2, lon2). It iteratively solves for lambda (difference in longitude on auxiliary sphere) until convergence, then returns distance s in meters.

Input parameters and where they come from

Parameter	Source in ATOS	Used for
lat1, lon1	GeoJSON point i from conf path	segment start distance
lat2, lon2	GeoJSON point i+1	segment end distance
truck lat/lon	CoT <point lat, lon> from truck	projection to path station
path_name	CoT <atos path_name="...">	choose trajectory dataset
path_index (optional)	CoT <atos path_index="...">	index-based ahead-gap mode

Outputs

- 1) Segment distance d_i (meters) between path points i and i+1
- 2) Cumulative station $s(i)$ along full path
- 3) Truck station s_{truck} after projection
- 4) Forward gap to next truck (meters), including wrap-around for closed-loop path

Pseudo-code (Vincenty + ATOS usage)

```
for each path segment (Pi, Pi+1):
    di = vincenty_inverse(Pi.lat, Pi.lon, Pi+1.lat, Pi+1.lon)
    s(i+1) = s(i) + di

for each incoming CoT truck point T(lat, lon):
    s_truck = project_to_polyline(T, path_points, cumulative_s)

sort connected trucks on same path by s_truck
for truck j:
    ahead = truck[(j+1) mod N]
    gap = ahead.s_truck - truck[j].s_truck
    if gap <= 0: gap += path_length

if gap <= 200: command = STOP
else if gap <= 500: command = SLOWDOWN
else apply hysteresis rules (300/700) for SLOWDOWN/RESUME
```

Flowchart A: CoT Data via TCP/TLS

Primary production path for trucks/pathfollower sending CoT over socket.

TCP listener active
startTcpServer() -> acceptTcpClients()

Client accepted
handleTcpClient(fd, peer)

TLS enabled?
if enabled: SSL_accept handshake

Receive bytes
recv()/SSL_read into buffer

Extract XML event
find <event> ... </event>

parseCotXml(xml)
uid, point lat/lon, track speed/course, path_name

Select trajectory
getTrajectoryForPath(path_name) fallback default

Project position
projectDistanceAlongTrajectory(lat, lon, ...)

Update truck state
trucks_[uid] fields + last_cot_message

Map uid->socket
uid_to_client_fd_[uid]=fd (+uid_to_ssl_)

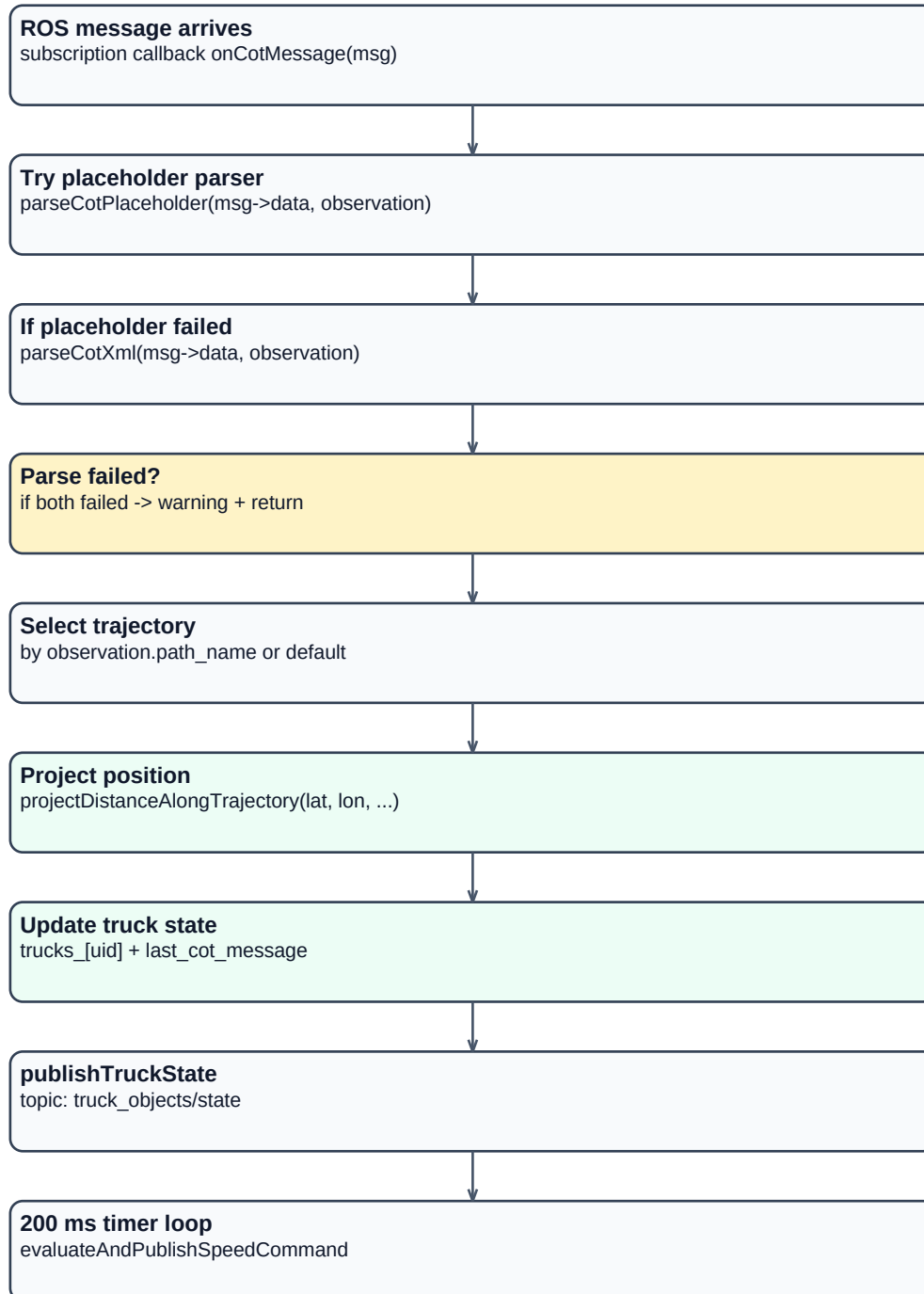
publishTruckState
topic: truck_objects/state

200 ms timer loop
evaluateAndPublishSpeedCommand -> sendSpeedCommandToTcpClient

TLS branch detail
cot_tls_enabled_? yes -> TLS path, no -> plain TCP

Flowchart B: CoT Data via ROS Topic

Compatibility/test ingress path on topic truck_objects/cot.



Parser order

1) parseCotPlaceholder 2) parseCotXml if both fail: return